

# REPOSITORY MIGRATION PLAN

ZCOS and ZAR rebuild from current ZedAI main

| Field            | Verified value                                                                      |
|------------------|-------------------------------------------------------------------------------------|
| Repository       | xoclonholdings/ZedAI                                                                |
| Audited baseline | main @ 7500b8010fdaa1783e5f745d74ca25af22825b31                                     |
| Baseline time    | August 7, 2026, 10:34 PM EDT (August 8, 2026, 02:34 UTC)                            |
| Plan status      | Required before rebuild mutations                                                   |
| Target authority | ZCOS Architecture Foundation + ZAR System Specification + ZAR behavior guidelines   |
| Design rule      | Preserve approved UI/UX; migrate ownership, routing, data, and runtime wiring first |

**Decision:** Do not begin a broad rewrite. Stabilize identity and side-effect boundaries first, establish canonical ZCOS authorities second, cut over one write authority at a time, and preserve the current visual system unless a separate design change is approved.

## 1. Purpose

This plan converts the current ZedAI repository into the locked ZCOS and ZAR architecture without treating existing code presence as proof of compliance. It is an implementation sequence, data migration contract, cutover strategy, and certification checklist. It does not authorize deletion of existing records, branch cleanup, production data movement, provider activation, or interface redesign by itself.

The repository is a migration source. The locked documents define the target. Where current code and the target disagree, the migration must preserve useful implementation, assign it to the correct canonical owner, verify the replacement path, block new writes to the superseded authority, and only then retire the old path.

## 2. Authority Order

1. ZCOS Architecture Foundation governs ecosystem ownership, eight galaxies, seven shared domains, partitions, Admin Access, Extensions, Settings, Portal, and migration boundaries.
2. The locked ZCOS Memory and Knowledge engine specifications govern their canonical records, lifecycle, provenance, retrieval, correction, deletion, and curation contracts.
3. ZAR System Specification governs ZAR Nexys, Operate, cognition, learning, Constitution, channels, orchestration, execution, and ZAR-specific acceptance requirements.
4. ZAR + User Interaction Guidelines and Behavior governs reasoning, communication, mobile-first development, file review, minimal changes, full-file delivery, and verification behavior.
5. Repository SPEC.md and current code provide dated implementation evidence. They do not override a locked target requirement.

**Recovered-state warning:** The 1,759-line updated SPEC.md created in the prior working session was not committed to GitHub and was removed from scratch by automated workspace maintenance. GitHub main still contains the August 4 specification. Phase 0 restores the canonical target before application mutations begin.

### 3. Audit Scope and Evidence

The plan is grounded in the three supplied governing PDFs and read-only inspection of current GitHub main. No repository files were modified. No build or test verdict is claimed because the repository checkout was no longer available after workspace maintenance.

| Evidence                     | Verified finding                                                                                                                                   | Migration consequence                                                                                                                 |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| GitHub main                  | Latest commit is 7500b80; current SPEC.md remains the August 4 implementation audit.                                                               | Restore canonical SPEC.md first and date all later implementation claims.                                                             |
| Branch inventory             | 29 remote branches are visible, including main, backup, multiple stabilization, agent, Claude, and legacy backup branches.                         | Branch cleanup is a separate reviewed operation; do not delete branches during rebuild setup.                                         |
| client/src/App.tsx           | Legacy top-level routes remain for trading, budget, workspaces, flows, projects, learning, connect, decisions, timeline, and discovery.            | Reassign routes to locked galaxy Desks without a one-step UI rewrite.                                                                 |
| rootManifests.ts             | Nexys still exposes eight legacy nodes: Identity, Memory, Knowledge, Workspaces, Projects, Tools, Connect, Settings.                               | Replace the information architecture with seven shared domains: Identity, Memory, Knowledge, Apps, Desk, Settings, Portal.            |
| NexysConversationSurface.tsx | Dock logic still implements Text, Talk, Image, Draw, Doc, and Upload behavior; Text currently opens ordinary chat.                                 | Preserve the dock visual pattern while rewiring to Text, Talk, Image, Chat, Document, Upload.                                         |
| GalaxyWorkspacePage.tsx      | Non-ZAR galaxies reuse ZAR's exact console shell, celestial scene, domains, and dock with theme tinting.                                           | Treat these routes as visual scaffolds; add each galaxy's own Console and specialized Desk contracts.                                 |
| galaxyConstellation.ts       | Eight stars exist, but ZETA is labeled SENTRY and current descriptions do not fully match locked ownership.                                        | Correct console/Desk metadata under a visual-preservation acceptance test.                                                            |
| conversations-crud.ts        | DELETE /api/conversations falls back to user_001 if authenticated ownership is absent.                                                             | Fail closed before any production data migration or deletion work.                                                                    |
| registerIntakeRoutes.ts      | Webhook verification becomes permissive when INTAKE_WEBHOOK_SECRET is unset; SMS accepts user_id from the inbound body.                            | Do not certify ZAR by Text until provider authenticity, identity binding, replay protection, and fail-closed production policy exist. |
| ExternalCommandGateway.ts    | Missing user_id becomes external:<channel>:<sender_id> and is persisted as task ownership.                                                         | External identifiers must remain sender claims, never ZCOS owners.                                                                    |
| object-memory-types.ts       | Object Memory mixes profile, projects, systems, features, decisions, preferences, rules, tasks, integrations, repositories, events, and conflicts. | Classify each object into Identity, Memory, Knowledge, Settings, Glow, Project/Desk, Files, or retirement.                            |

| Evidence                   | Verified finding                                                                                                                                                 | Migration consequence                                                                                           |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| KnowledgeCurationEngine.ts | Knowledge curation reads MemoryService stores and writes JSON reports under hub/shared-memory/curation.                                                          | Preserve curation logic but move authority to the canonical Knowledge Engine and PostgreSQL-backed audit state. |
| ZarReflectionEngine.ts     | Reflection writes summaries directly into project_memory.                                                                                                        | Convert reflection into governed learning or Memory candidates; stop direct active-memory writes.               |
| MemoryInjector.ts          | Runtime context still loads shared filesystem working, episodic, consensus, approval, and foundation files.                                                      | Replace with authorized canonical projections; filesystem material remains migration evidence, not authority.   |
| migrations.ts              | Current tables include core_memory, project_memory, scratchpad_memory, trading_state, and learning_state but not the locked partition/grant/Constitution models. | Add new schema behind adapters before moving writers or data.                                                   |

## 4. Non-Negotiable Migration Rules

- Preserve the approved interface design. Do not change layout, spacing, styling, hierarchy, motion, or interaction patterns unless separately authorized.
- No blind rewriting. Inspect each file and behavior-affecting dependency before implementation.
- No invented owners. Missing authenticated identity fails closed.
- One canonical authority per record class. Adapters may coexist temporarily; parallel authoritative writers may not.
- PostgreSQL is production authority for canonical ZCOS records and ZAR learning/Constitution state. JSON, Chroma, caches, reports, exports, and vectors are projections or migration sources only.
- Every migrated record retains original ID when safe, legacy ID otherwise, owner, originating galaxy, source, lifecycle, version, and migration batch provenance.
- Every cutover has shadow verification, divergence reporting, rollback evidence, and a blocked-new-write step for the superseded path.
- No deletion is authorized by classification alone. Existing personal, runtime, admin, or branch material requires a separate retention and removal decision.
- Completion requires live-path evidence. A route, UI, service, adapter, prompt, or test fixture by itself is not proof of a working feature.

## 5. Canonical Ownership Map

| Current area           | Locked owner                                     | Disposition                                                                  |
|------------------------|--------------------------------------------------|------------------------------------------------------------------------------|
| Nexus/Nexys root shell | ZAR Nexys + seven shared domains                 | Preserve shell and scene; replace legacy root-node ownership and routes.     |
| Workspaces             | Galaxy Desks                                     | Classify each workspace; remove Workspaces as a ZAR shared-domain authority. |
| Projects               | Originating galaxy Desk; All Projects projection | Add ownerUserId and originGalaxyId; preserve project history.                |

| Current area                   | Locked owner                                                   | Disposition                                                                       |
|--------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Tools                          | Apps, Capabilities, or ZYLO Automate                           | Separate installed Extensions, capability permissions, and automation artifacts.  |
| Connect                        | Settings -> Integrations                                       | Retain backend connectors; move user-facing control surface and scope model.      |
| Object Memory                  | Shared object framework + canonical domain owners              | Split mixed objects and retire undifferentiated authority.                        |
| Knowledge Ingestion + Curation | ZCOS Knowledge Engine                                          | Unify graph, source, lifecycle, curation, Lexicon, and retrieval authority.       |
| core/project/scratchpad memory | ZCOS Memory, temporary context, Project records, or retirement | Classify; do not rename these stores into compliance.                             |
| Reflection                     | ZAR learning candidates / Memory candidates                    | Stop direct project_memory writes.                                                |
| Learning Studio                | ZENITH Logos -> Scholar                                        | Keep education and mastery separate from relationship learning.                   |
| Flows/Runs/Suggestions         | ZYLO Compass -> Automate                                       | Preserve engines; move ownership and invoke through typed grants.                 |
| Trading/Budget                 | ZILLION Prosper -> Capital                                     | Preserve governed logic; remove top-level ZAR ownership.                          |
| ZYNC Coding Operator           | ZYNC Canvas -> Build                                           | Preserve cross-galaxy coordination through Admin Access.                          |
| Files/uploads                  | ZENITH Logos -> Scholar -> Files                               | Original artifacts move to Files; extraction creates linked Knowledge candidates. |
| Discovery                      | ZWAP! Discovery -> Explore                                     | Map Glow, News, and Journal/Blog to the locked Desk.                              |
| External intake                | ZCOS Channel Service + verified Identity binding               | Normalize sender claims; never derive ownership from provider input.              |

## 6. Migration Phases

### Phase 0 - Authority Recovery and Frozen Baseline

**Objective:** Restore the locked target into the repository and create a reproducible evidence baseline before application changes.

#### Required changes

- Recreate the canonical SPEC.md from the locked documents and the verified current implementation inventory; preserve dated status language and current audit exceptions.
- Record baseline commit 7500b80, branch inventory, schema inventory, route registry, writers, scheduled jobs, runtime files, and user-facing routes.
- Create machine-generated inventories for routes, database tables, filesystem writers, provider adapters, prompts, background timers, and UI entry points.
- Define migration status vocabulary: preserve, adapt, migrate, replace, retirement-blocked, retire-later, and certify-later.

#### Required evidence

- Canonical SPEC.md diff reviewed against all locked source documents.
- Inventory output is reproducible from one command and tagged with commit SHA and date.
- No application code, data, UI, or branches changed in this phase.

**Exit gate:** SPEC.md is approved as repository authority and the baseline inventory has no unexplained writers or top-level routes.

**Rollback boundary:** Documentation-only commit can be reverted independently; the audited 7500b80 source state remains unchanged.

## Phase 1 - Identity, Ownership, and Side-Effect Stabilization

**Objective:** Eliminate fallback identity and unsafe external-action paths before migrating personal or cross-galaxy data.

### Required changes

- Introduce one authenticated OwnerContext contract used by protected routes, services, background jobs, channels, and migrations.
- Remove user\_001, anonymous, user, unknown, default-user, sender-derived, and request-body ownership fallbacks; return explicit authentication errors.
- Require production webhook authenticity. Add provider signature verification, timestamp tolerance, replay cache, message idempotency, safe retry, and redacted logging.
- Audit every outbound messaging and side-effect path against action-specific approval immediately before execution.
- Add authorization and ownership regression tests before changing schemas or data.

### Required evidence

- Static scan reports zero prohibited fallback owners in executable paths.
- Cross-user, missing-user, forged-sender, replay, duplicate-message, expired-signature, and changed-scope tests pass.
- Delete-all conversations proves authenticated ownership and refuses missing identity without deleting any row.

**Exit gate:** No protected read, write, deletion, task, approval, or channel path can operate without one verified ZCOS owner.

**Rollback boundary:** Keep old routes deployed but fail closed; do not restore permissive fallback behavior during rollback.

## Phase 2 - Shared ZCOS Data Foundation

**Objective:** Create the database and service contracts that Memory, Knowledge, Projects, Files, grants, and audits can share without collapsing authority.

### Required changes

- Add canonical Identity reference, galaxy registry, partition registry, grants, provenance, audit, migration batch, source, and version models.
- Define the shared object envelope with ownerUserId, originGalaxyId, authorityKind, type, canonicalName, lifecycleState, source links, version, timestamps, and legacy identifiers.
- Implement explicit, scoped, revocable, time-bounded, auditable Admin Access grants for cross-galaxy read, write, and contribution authority.
- Add repositories for canonical records and projections; local JSON and vectors remain indexes or migration sources.
- Create adapters so current callers can read the new contracts without moving authoritative writes yet.

#### Required evidence

- Database migration forward and rollback tests pass on empty and representative legacy datasets.
- Partition and grant tests prove deny-by-default cross-galaxy behavior.
- Audit records correlate owner, request, source, grant, mutation, and outcome.

**Exit gate:** Shared foundation is deployed with no user-facing change and no second authoritative writer.

**Rollback boundary:** Disable new adapters and roll back schema only while no canonical writes have been accepted.

### Phase 3 - Canonical Memory Engine

**Objective:** Evolve Object Memory into the structural foundation of one ZCOS Memory Engine with eight partitions and governed lifecycle.

#### Required changes

- Implement source ledger, canonical memory records, relationship links, lifecycle transitions, retrieval indexes, audit ledger, toggle enforcement, and deletion cascade jobs.
- Support experience, decision, person/relationship, event, and user-directed memory types with occurredAt and confirmation method.
- Classify legacy Object Memory, core\_memory, project\_memory, scratchpad, foundation exports, and reflection records; route non-memory objects to their proper owners.
- Run idempotent migration batches with quarantine for ambiguous ownership, type, provenance, or galaxy origin.
- Dual-read in shadow mode, compare results, cut over canonical reads, then block legacy writes before retiring adapters.

#### Required evidence

- You, Topics, and Galaxies resolve the same canonical record IDs rather than duplicate stores.
- Memory off creates no long-term memory and retrieves none while preserving existing records.
- Correction, supersession, rejection, and forgetting cascade through every derived index and context projection.
- Cross-galaxy Memory access fails without an active grant and appears in audit evidence when granted.

**Exit gate:** Canonical Memory passes the locked engine acceptance suite and every legacy writer is blocked or explicitly temporary.

**Rollback boundary:** Re-enable read adapters only; never re-enable parallel legacy writes after canonical cutover.

## Phase 4 - Canonical Knowledge Engine

**Objective:** Unify Object Memory knowledge objects, Knowledge Ingestion, Memory-backed Curation, Lexicon, and retrieval under one partitioned Knowledge authority.

### Required changes

- Implement Topics, Knowledge Map, Sources, Lexicon, and Curation over canonical Knowledge records and eight partitions.
- Preserve deterministic extraction, evidence, relationships, conflict detection, health scoring, source diversity, freshness, and Lexicon senses after mapping them to canonical contracts.
- Move Curation authority from MemoryService inputs and hub JSON outputs to canonical Knowledge records and auditable review jobs; reports remain projections.
- Route original artifacts to ZENITH Files and derived text/claims to Knowledge with source/version links.
- Replace keyword-only or status-unsafe retrieval with lifecycle, owner, partition, source, confidence, and authorization gates; vector search remains an index.

### Required evidence

- Duplicate, conflicting, superseded, rejected, historical, and under-review objects cannot appear as one flattened truth.
- Every promoted claim has source origin, evidence, lifecycle, owner, partition, and curation decision.
- Object Memory and Knowledge Ingestion no longer create competing canonical objects.

**Exit gate:** One Knowledge authority serves all five surfaces and eight partitions with state-safe retrieval.

**Rollback boundary:** Restore legacy readers as read-only diagnostic sources; do not restore legacy promotions or writers.

## Phase 5 - Governed ZAR Runtime, Learning, and Constitution

**Objective:** Enforce one no-bypass ZAR response path and restore the confirmed relationship-learning system without duplicating Memory or Knowledge.

### Required changes

- Create ZarRuntime and supporting Behavior, Context, Response, Trace, Learning, and Constitution service contracts from the locked specification.
- Make POST /api/orchestrate the only user-visible response path; route streaming, specialists, channels, and legacy callers through identical gates.

- Compile behavior principles into executable checks and fixtures; keep concise, direct, mobile-readable presentation separate from hidden reasoning.
- Replace ZarReflectionEngine direct project\_memory writes with provenance-linked Experience, Observation, Proposal, or Memory candidate creation.
- Add PostgreSQL records for experiences, evidence, observations, proposals, review transitions, ConstitutionArticle, Amendment, DeletionRequest, sections, versions, and atomic resolution.
- Remove canned/template output that can impersonate ZAR; retain only clearly labeled system errors and blocked states.

#### Required evidence

- Direct-first, evidence, confluence, context, state, uncertainty, rationale, and outcome-learning fixtures pass across chat, research, tasks, errors, and channels.
- Only confirmed Constitution Articles enter active collaboration context; rejected and pending states remain excluded.
- Forced transaction failure proves proposal resolution and Constitution mutation roll back together.
- No response route bypasses governed presentation or leaks prompts, chain-of-thought, secrets, provider traces, or retrieval chunks.

**Exit gate:** One governed ZAR runtime produces every live user response and the learning/Constitution pipeline passes end to end.

**Rollback boundary:** Return callers to the prior governed ChatExecutionService adapter while preserving canonical records; never return to direct reflection writes.

## Phase 6 - Projects, Files, and Operate Desk

**Objective:** Map work into Support, Brainstorm/Research, and Tasks (Implement) while preserving project and artifact ownership.

#### Required changes

- Add project originGalaxyId, ownerUserId, instructions, goals, task, decision, artifact, source, activity, and cross-galaxy reference contracts.
- Create OperateService, ResearchService, and TaskService adapters around existing project, research, execution, and approval foundations.
- Move original uploads and generated artifacts to ZENITH Files; keep conversation and Project references stable.
- Make upload success, extraction success, ingestion success, and canonical promotion distinct observable states.
- Map existing workspace behaviors to an Operate surface or another locked galaxy Desk before removing any route.

#### Required evidence

- Representative Support, Research, and Implement scenarios complete with correct owner, galaxy, Project, source, approval, artifact, and verification records.
- Upload tests cover partial extraction, failed ingestion, duplicate version, unsupported type, deletion, and source traceability.



- All Projects is a projection and does not erase origin galaxy ownership.

**Exit gate:** Operate functions end to end without project-local Memory/Knowledge copies or file ownership ambiguity.

**Rollback boundary:** Keep legacy route adapters mapped to the same canonical Project/File records until mobile navigation cutover is verified.

## Phase 7 - Seven Domains and Eight Galaxy Shells

**Objective:** Align the visible information architecture with the locked galaxy/domain model while preserving approved visual design.

### Required changes

- Replace Nexys legacy root manifests with Identity, Memory, Knowledge, Apps, Desk, Settings, and Portal.
- Convert Workspaces, Projects, Tools, and Connect from top-level Nexys domains into Desk/Projects, Apps/Capabilities, ZYLO Automate, and Settings -> Integrations as appropriate.
- Retain the eight-star constellation positions, proportions, imagery, motion, and vessel treatment; correct console/Desk metadata and destination ownership.
- Stop using ZAR's exact domain scene as the completed implementation for every galaxy; provide galaxy-specific Console identity and one specialized Desk contract behind the shared shell.
- Preserve the current dock appearance while changing the baseline to Text, Talk, Image, Chat, Document, Upload. Text opens ZAR by Text; Chat opens Nexys conversation; Draw is removed from the final baseline.
- Implement Portal as transport only and move system-wide controls to ZCOS Command Desk.

### Required evidence

- Every galaxy exposes the same seven domain purposes and exactly one specialized Desk.
- Routes, labels, deep links, back behavior, focus order, and reduced-motion behavior pass on the supported iPhone viewport.
- No visual-regression diff is accepted without explicit design approval; ownership changes are verified separately from appearance.

**Exit gate:** The visible architecture matches the locked model and all existing useful routes have an explicit destination or retirement adapter.

**Rollback boundary:** Feature-flag the new manifests and routes; revert routing while retaining canonical backend ownership.

## Phase 8 - Galaxy Feature Reassignment

**Objective:** Move specialized systems to their locked galaxy Desks without duplicating engines or fragmenting ZAR identity.

### Required changes

- Move Flows, Runs, Suggestions, Skills, and Templates to ZYLO Compass -> Automate.
- Move ZYNC Coding Operator, coding, design, and publishing functions to ZYNC Canvas -> Build.
- Move monitoring, logs, and diagnostics to ZETA Control -> Integrity.
- Move threads, notes, and rooms to ZENO Unite -> Forum.
- Move Glow, News, and Journal/Blog to ZWAP! Discovery -> Explore.
- Move Learning Studio, Library, and Files to ZENITH Logos -> Scholar.
- Move Budgeting, Trading, and Investing to ZILLION Prosper -> Capital; live trading remains blocked until separate certification.
- Expose cross-galaxy invocation to ZAR only through typed capabilities and Admin Access grants.

### Required evidence

- Each moved service retains owner, source, history, permissions, and legacy deep-link compatibility during transition.
- Direct access and ZAR-coordinated access produce the same canonical records and audit events.
- No moved subsystem remains a hidden ZAR-owned canonical writer.

**Exit gate:** All specialized systems have one galaxy owner and approved cross-galaxy invocation contracts.

**Rollback boundary:** Route legacy links through compatibility adapters; do not duplicate data back into ZAR.

## Phase 9 - ZAR by Text and Foreground Voice

**Objective:** Activate SMS and foreground voice as channels to the same governed ZAR only after identity and runtime foundations are certified.

### Required changes

- Implement provider-independent ChannelService and phone-to-Identity binding with explicit consent, revocation, unlinking, and privacy controls.
- Replace generic SMS intake ownership with verified bindings; unverified or ambiguous senders fail closed without account enumeration.
- Add signature authenticity, rate limit, replay protection, message idempotency, safe retry, redaction, STOP/START/HELP/STATUS/UNLINK, and sensitive-action confirmation.
- Route SMS through ZarRuntime and apply concise SMS presentation without creating a separate chatbot, Memory, Knowledge, or orchestration path.
- Implement foreground saying-'ZAR' activation while the iPhone app is open, with visible permission, listening, recording, transcription, cancellation, failure, and retention states.
- Do not represent locked-screen or app-closed activation as available in the first release.

### Required evidence

- Smartphone and basic SMS-capable phone flows pass linking, consent, identity, reply, revocation, replay, retry, idempotency, rate-limit, redaction, and action-confirmation suites.
- Nexys, SMS, and voice resolve the same Identity, behavior policy, Memory/Knowledge authority, Projects, approvals, and audit trail.
- Foreground wake-word activation passes supported iPhone accessibility and degraded-state tests.

**Exit gate:** Channels are certified as transports into the same ZAR and cannot broaden identity or action authority.

**Rollback boundary:** Disable the channel adapter and preserve conversation/audit records; do not fall back to generic sender-derived ownership.

## Phase 10 - Retirement, Recovery, and Production Certification

**Objective:** Remove superseded authorities only after evidence proves canonical operation and recovery.

### Required changes

- Block all legacy writers, monitor read divergence through a defined observation window, and reconcile quarantined records.
- Retire obsolete adapters, filesystem authorities, duplicate graphs, stale routes, and prompts in small reviewed changes.
- Review tracked runtime/personal/admin artifacts and legacy zed-memory under a separate retention, ownership, migration, and deletion decision.
- Complete branch cleanup only through a separately approved inventory and recoverability plan.
- Run production certification across identity, partitions, grants, behavior, Memory, Knowledge, Projects, Files, Desks, channels, actions, mobile, accessibility, recovery, backup, and audit.

### Required evidence

- Certification package records commit, build, environment, database migration version, fixtures, results, exceptions, approver, date, and unresolved blockers.
- Backup restore, failed migration resume, rollback, deletion cascade, partial provider action, and incident reconciliation drills pass.
- No legacy authority receives new writes and no production surface depends on an uncertified adapter.

**Exit gate:** The repository is production-certified against the locked specifications, with remaining blocked capabilities labeled truthfully.

**Rollback boundary:** Rollback uses the last certified application build and forward-compatible canonical data; destructive reverse migrations are prohibited without a tested recovery procedure.

## 7. Data Migration Contract

| Requirement    | Implementation rule                                                                                                          |
|----------------|------------------------------------------------------------------------------------------------------------------------------|
| Batch identity | Every run has migrationBatchId, source commit, schema version, startedAt, completedAt, actor, counts, checksum, and outcome. |
| Idempotency    | Re-running the same batch cannot duplicate canonical records, relationships, sources, audit events, or side effects.         |
| Ownership      | A record without a verified owner is quarantined. No fallback, sender-derived, or shared default owner is permitted.         |

| Requirement     | Implementation rule                                                                                                                                     |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Galaxy origin   | Use evidence from source route, workspace, Project, agent, or feature. Ambiguous origin is quarantined rather than guessed.                             |
| Legacy identity | Preserve safe source IDs; otherwise retain legacyAuthority, legacyRecordId, and legacySourcePath.                                                       |
| Provenance      | Retain source type, source ID, file/conversation/action reference, extraction/migration time, and evidence necessary to explain the canonical record.   |
| Lifecycle       | Do not promote archived, rejected, conflicting, proposed, or superseded source records to active canonical state without an explicit rule and evidence. |
| Files           | Copy/transfer originals with checksums and versions; extraction and Knowledge candidates remain linked derivatives.                                     |
| Validation      | Pre-counts, post-counts, checksums, sample equivalence, relationship integrity, authorization, lifecycle, and retrieval comparisons are required.       |
| Quarantine      | Ambiguous, malformed, ownerless, conflicting, or unsupported records remain immutable in a review queue with reason and source.                         |
| Cutover         | Shadow read -> compare -> canonical read -> block legacy write -> monitor -> retire. Never enable dual authoritative writes.                            |
| Deletion        | Migration does not authorize deletion. Forgetting and privacy deletion use canonical cascades after source ownership is resolved.                       |

## 8. Required Service and Schema Packages

| Package        | Minimum contracts                                                                                  | First consumers                            |
|----------------|----------------------------------------------------------------------------------------------------|--------------------------------------------|
| ZCOS Core      | OwnerContext; GalaxyRegistry; PartitionService; AdminAccessService; AuditService; MigrationService | All protected routes and engines           |
| Shared Objects | ObjectEnvelope; SourceRef; Relationship; Version; Lifecycle; LegacyRef                             | Memory and Knowledge                       |
| Memory         | MemoryService; MemoryLifecycle; MemoryRetrieval; MemoryIndex; MemoryDeletion                       | ZAR context and Memory UI                  |
| Knowledge      | KnowledgeService; SourceService; CurationService; LexiconService; KnowledgeRetrieval               | Research, uploads, Knowledge UI            |
| ZAR Core       | ZarRuntime; ZarBehaviorPolicy; ZarContextService; ZarResponseService; ZarTraceService              | Nexys, SMS, voice, specialists             |
| ZAR Learning   | ZarLearningService; ZarConstitutionService                                                         | Outcome handling and collaboration context |
| Operate        | OperateService; ResearchService; TaskService; ProjectService                                       | Support, Brainstorm/Research, Implement    |
| Files          | FileService; FileVersionService; ExtractionService; ArtifactReferenceService                       | Uploads, Projects, ZENITH Scholar          |
| Channels       | ChannelService; IdentityBindingService; InboundAuthenticityService; ConsentService                 | SMS, voice, email, messaging               |

| Package   | Minimum contracts                                                            | First consumers                |
|-----------|------------------------------------------------------------------------------|--------------------------------|
| Execution | CapabilityRegistry; ApprovalService; ExecutionService; ReconciliationService | Tasks and cross-galaxy actions |

## 9. Verification and Certification Matrix

| Layer       | Minimum verification                                                                           | Blocks release when                                                             |
|-------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Static      | Typecheck, lint, prohibited-owner scan, route/writer inventory, schema diff                    | Fallback owner, bypass path, duplicate authority, or unexplained writer remains |
| Unit        | Lifecycle, authorization, partition, provenance, idempotency, policy, classification           | State transition or deny-by-default behavior is unproved                        |
| Integration | Database transactions, adapters, ingestion, retrieval, approvals, channel authenticity         | Canonical and legacy paths diverge without explanation                          |
| Data        | Counts, checksums, source links, owner isolation, relationship integrity, quarantine           | Records are lost, duplicated, ownerless, or silently promoted                   |
| Runtime     | POST /api/orchestrate live path, specialists, tools, error and recovery states                 | A response or action bypasses policy or verification                            |
| UI          | Supported iPhone viewport, touch, dynamic type, screen reader, focus, contrast, reduced motion | Primary flow fails or approved design drifts                                    |
| Channels    | SMS and voice identity, consent, authenticity, replay, retry, action confirmation              | Channel can retrieve or act without verified binding                            |
| Operational | Deployment, scheduler ownership, backup/restore, migration resume, rollback, audit export      | Unknown outcomes cannot be reconciled safely                                    |
| Security    | Cross-user, cross-galaxy, untrusted content, secret leakage, provider scope, deletion          | Unauthorized access, prompt override, or scope broadening succeeds              |

## 10. Commit, Branch, and Release Discipline

- Use one migration concern per reviewed commit or narrowly related commit set. Keep documentation, schema, adapters, data migration, routing, and retirement changes separable.
- Do not delete or rewrite the 29 visible remote branches during the migration. Produce a separate branch disposition report with last commit, unique commits, merged status, owner, backup value, and recovery path.
- Each phase begins from current main, records the baseline SHA, updates SPEC.md status only after verification, and leaves a rollback tag or release reference.
- Schema changes are expand -> backfill -> verify -> cut over -> contract. Never combine destructive contraction with first deployment of the replacement.
- Feature flags may control new readers, manifests, routes, and channel adapters. They may not restore insecure ownership fallbacks or parallel canonical writers.

- Production claims require the certification evidence package. Planned, scaffolded, partial, active, blocked, and certified remain distinct statuses.

## 11. First Implementation Package

**Start here:** The first code package is not Memory or the new UI. It is Phase 0 plus the fail-closed ownership slice of Phase 1.

1. Restore and commit the canonical SPEC.md alone.
2. Add the reproducible route, writer, store, branch, and runtime-artifact inventory script and baseline report.
3. Introduce OwnerContext without changing visible UI.
4. Fix DELETE /api/conversations so missing owner returns 401/403 and no deletion occurs.
5. Fix ExternalCommandGateway so an external sender claim cannot become task owner; unbound sender requests remain unauthenticated channel events only.
6. Make production intake reject requests when authenticity configuration is absent; add signature/replay/idempotency contract tests.
7. Run typecheck, tests, build, ownership regression suite, and live API checks; update SPEC.md with exact verified results and date.

## 12. Completion Definition

Migration is complete only when the repository implements the locked ZCOS and ZAR contracts through one authenticated, partitioned, provenance-preserving, lifecycle-safe system; every legacy authority is blocked from new writes or explicitly retained as a non-authoritative source; the approved mobile interface remains intact; and the production certification package proves the live paths.

Capabilities that remain externally dependent or separately governed - including provider channels, autonomous schedulers, destructive workflows, and live trading - must remain labeled partial, blocked, or uncertified until their specific suites pass. The system must report those states truthfully.

## Appendix A - Verified Source Register

| Source                                                          | Role                                                                                                        |
|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| ZAR + User Interaction Guidelines and Behavior (August 4, 2026) | Normative execution, reasoning, communication, mobile, code, and workflow behavior                          |
| ZCOS Architecture Foundation (August 7, 2026)                   | Locked ecosystem architecture, galaxy/domain ownership, partitions, Settings, Portal, and rebuild rules     |
| ZAR System Specification (August 8, 2026)                       | Locked ZAR Nexys, Operate, learning, Constitution, channels, runtime, migration, and certification contract |
| xoclonholdings/ZedAI main @ 7500b80                             | Current implementation evidence inspected read-only on August 8, 2026                                       |

| Source                           | Role                                                                                   |
|----------------------------------|----------------------------------------------------------------------------------------|
| Current GitHub SPEC.md @ c73394f | August 4 implementation inventory and known exceptions; superseded as target authority |
| Current remote branch inventory  | 29 visible branches; cleanup not authorized by this plan                               |

## Appendix B - Immediate Known Blockers

- Canonical updated SPEC.md is not present on GitHub main.
- Fallback and sender-derived ownership exists in executable paths.
- External webhook verification can be permissive when configuration is absent.
- Object Memory, Knowledge Ingestion, MemoryService-backed Curation, project\_memory Reflection, and filesystem context form overlapping authorities.
- The current database schema lacks the locked partition, grant, lifecycle, provenance, learning, and Constitution models.
- The current Nexys root manifests and dock do not match the locked seven domains and final six actions.
- Non-ZAR galaxy workspaces are themed ZAR scaffolds rather than completed galaxy-specific Console/Desk implementations.
- Repository build and tests were not rerun during this plan because workspace maintenance removed the local checkout; current code presence is not a fresh certification result.